

RAPPORT 'PROJET ORCHESTRATION DU NBA_PREDICTOR'

Infrastructure et orchestration de données

Cours de Ketsia Mulapi Tita

Maël M. ZINSOU

1. Introduction et contexte

L'industrialisation des applications de Machine Learning (ML) est un enjeu critique. Au-delà de la performance du modèle, garantir sa **déployabilité, maintenabilité et supervision** est indispensable. Ce projet transforme une application locale de classification NBA en un **pipeline industriel complet**, reproductible et observable via l'écosystème Data/MLOps (Docker, Kubernetes, Airflow, Prometheus, Grafana).

Déployé localement via Minikube pour simuler la production, le projet répond aux problématiques suivantes :

- **Conteneurisation** : Comment isoler proprement une application ML ?
- **Orchestration** : Comment piloter les composants et automatiser les appels API ?
- **Observabilité** : Comment superviser l'infrastructure et les métriques métier ?
- **Résilience** : Comment diagnostiquer et résoudre les erreurs d'une architecture distribuée ?

L'architecture s'articule autour de trois flux : **Applicatif** (Interaction utilisateur), **Orchestration** (Pilotage Airflow) et **Observabilité** (Monitoring temps réel).

2. Choix technologiques et justification

La stack technique privilégie des outils standards et éprouvés pour garantir cohérence et reproductibilité.

Technologie	Rôle	Justification du choix
Docker	Conteneurisation	Isolation des dépendances, comportement identique dev/prod.
K8s (Minikube)	Orchestration	Gestion du cycle de vie, réseaux, namespaces. Standard industriel.
FastAPI	Backend ML	Performance, doc auto (OpenAPI), intégration native des métriques.
Nginx	Frontend / Proxy	Serveur statique performant et Reverse Proxy (gestion CORS).
Apache Airflow	Orchestration	Planification des tâches, logs centralisés, pilotage du pipeline.
PostgreSQL	Métatada DB	Base robuste pour Airflow (image officielle postgres:16).
Prometheus	Collecte métriques	Standard du monitoring K8s, scraping automatique.
Grafana	Visualisation	Dashboards clairs pour l'infra et les métriques métier.

3. Architecture globale

L'architecture, déployée sur Minikube, est modulaire et cloisonnée par **namespaces** pour séparer les responsabilités.

La figure ci-dessous présente le **schéma d'architecture global** du projet, réalisé à l'aide de l'outil Excalidraw.

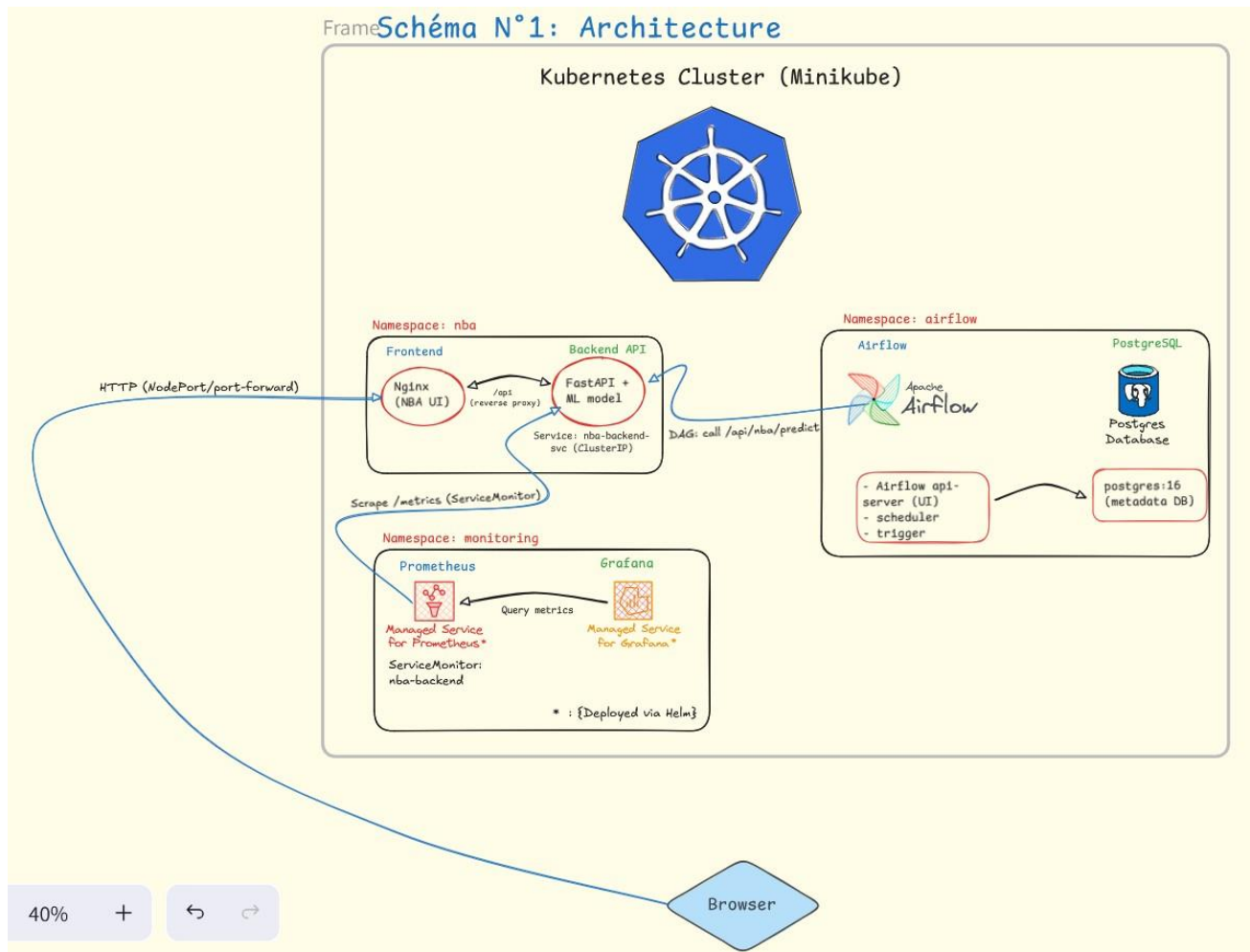


Figure 1 – Architecture globale du pipeline NBA (voir schéma Excalidraw)

3.1 Vue d'ensemble du cluster Kubernetes

Le cluster Kubernetes constitue le socle de l'architecture. Il assure :

- l'orchestration des conteneurs,
- la communication réseau entre les services,
- l'isolation logique des composants via des namespaces,
- l'intégration avec les outils d'observabilité et d'orchestration.

Trois namespaces principaux structurent l'architecture :

- nba : application métier (frontend et backend),
- airflow : orchestration des traitements,
- monitoring : supervision et métriques.

Cette organisation permet de cloisonner les usages tout en conservant une communication maîtrisée entre les composants.

3.2 Namespace nba – Application métier

Le namespace nba regroupe l'ensemble des composants directement liés à l'application de classification NBA.

Frontend – Interface utilisateur

Le frontend est basé sur un serveur **Nginx**, chargé de servir l'interface web statique de l'application. Il joue également le rôle de **reverse proxy**, redirigeant les requêtes `/api/*` vers le backend.

Ce choix permet :

- d'éviter les problématiques de CORS côté navigateur,
- de masquer l'implémentation interne du backend,
- de proposer un point d'entrée unique à l'utilisateur.

Le frontend est déployé via un Deployment Kubernetes et exposé localement à l'aide d'un Service de type NodePort ou par port-forward.

Backend – API de prédiction

Le backend est une API développée avec **FastAPI**, intégrant le modèle de machine learning chargé de prédire la pertinence du recrutement d'un joueur NBA.

Il expose plusieurs endpoints, notamment :

- des endpoints métier pour la prédiction et l'analyse,
- un endpoint `/metrics` destiné à l'observabilité.

Le backend est déployé via un Deployment Kubernetes et exposé en interne par un Service de type **ClusterIP**. Les communications avec le frontend, Airflow et Prometheus reposent sur la résolution DNS interne de Kubernetes.

3.3 Namespace airflow – Orchestration des traitements

Le namespace airflow est dédié à l'orchestration des traitements métier à l'aide d'Apache Airflow.

Airflow

Airflow est déployé dans Kubernetes via Helm, avec une configuration adaptée à un environnement local :

- utilisation du **LocalExecutor**,
- séparation des composants (API server, scheduler),
- persistance des métadonnées.

Airflow permet de définir des **DAGs** (Directed Acyclic Graphs) décrivant les tâches à exécuter. Dans ce projet, un DAG spécifique pilote des appels vers l'API backend afin de démontrer l'orchestration d'un pipeline de prédiction.

Base de données PostgreSQL

Une base de données PostgreSQL dédiée est utilisée pour stocker les métadonnées d'Airflow (état des DAGs, exécutions, logs).

Elle est déployée à partir de l'image officielle `postgres:16` et associée à un **Persistent Volume Claim**, garantissant la persistance des données.

3.4 Namespace monitoring – Supervision et métriques

Le namespace monitoring regroupe les outils d'observabilité du système.

Prometheus

Prometheus est chargé de collecter les métriques du cluster Kubernetes ainsi que les métriques applicatives exposées par le backend.

La collecte des métriques de l'API NBA est réalisée via un **ServiceMonitor**, qui définit automatiquement les cibles à scraper à intervalles réguliers.

Grafana

Grafana est utilisé pour la visualisation des métriques collectées par Prometheus.

Il permet d'afficher :

- des dashboards génériques sur l'état du cluster,
- des métriques spécifiques à l'API de prédiction (nombre de requêtes, latence).

Grafana constitue l'interface principale de supervision du pipeline.

3.5 Flux de communication inter-composants

L'architecture repose sur plusieurs types de flux clairement identifiés :

- **flux applicatif** : navigateur → frontend → backend,
- **flux d'orchestration** : Airflow → backend,
- **flux d'observabilité** : Prometheus → backend, Grafana → Prometheus.

Ces flux sont rendus possibles par les Services Kubernetes et la résolution DNS interne du cluster, sans exposition directe inutile des composants sensibles.

3.6 Intérêt de l'architecture proposée

Cette architecture illustre une approche réaliste de l'industrialisation d'une application de machine learning. Elle démontre la capacité à :

- structurer un projet complexe,
- orchestrer des traitements métier,
- superviser une application distribuée,
- tout en conservant une solution reproductible et démontrable en environnement local.

4. Description du pipeline NBA

Le fonctionnement du pipeline repose sur l'interaction de trois flux complémentaires.

Le **schéma de flux** présenté ci-dessous illustre les différentes étapes du pipeline ainsi que les interactions entre les composants.

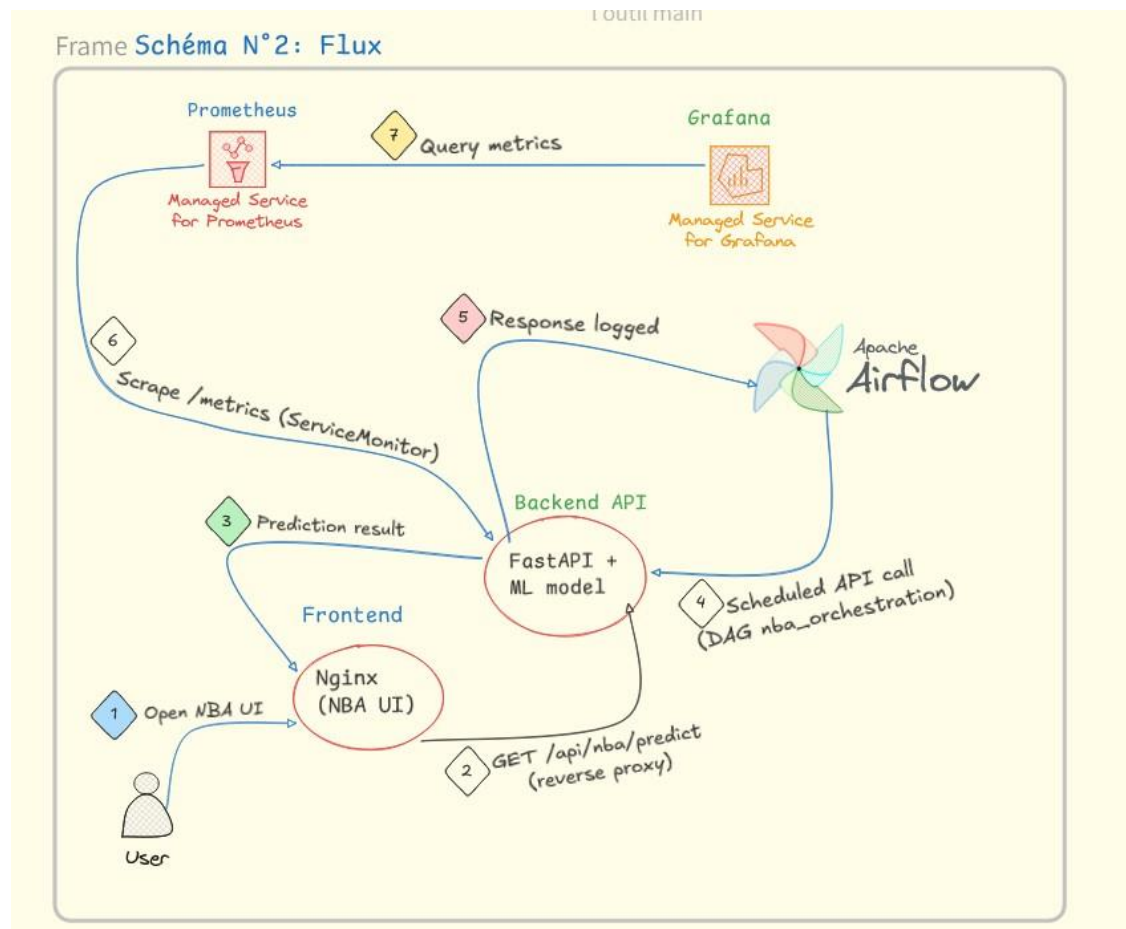


 Figure 2 – Flux de fonctionnement du pipeline NBA

4.1 Flux applicatif (Utilisateur)

Scénario classique d'utilisation web :

1. L'utilisateur accède à l'UI via le navigateur.
2. **Nginx** sert l'interface et redirige les appels API (/api/*) vers le backend.
3. **FastAPI** exécute le modèle de prédiction et retourne le résultat. *Intérêt* : Découplage frontend/backend et expérience utilisateur fluide.

4.2 Flux d'orchestration (automatisation via Airflow)

Automatisation des traitements via le DAG nba_orchestration :

1. Déclenchement (manuel ou planifié) du DAG dans Airflow.
2. Appel HTTP automatisé vers l'endpoint /api/nba/predict.
3. Enregistrement de la réponse et des logs d'exécution. *Intérêt* : Simulation de traitements batch ou de scoring périodique.

4.3 Flux d'observabilité (Monitoring)

Surveillance temps réel :

1. **Exposition** : FastAPI expose ses compteurs sur /metrics.
2. **Collecte** : Prometheus scrape ces données via un ServiceMonitor.
3. **Visualisation** : Grafana affiche la latence, le volume de requêtes et l'impact des jobs Airflow. *Intérêt* : Visibilité indispensable pour le diagnostic et le suivi de performance.

5. Déploiement et mise en œuvre technique

L'organisation du projet sépare strictement le code applicatif, les manifestes Kubernetes, les DAGs et la configuration Helm.

5.1 Conteneurisation (Docker)

Les images sont construites directement dans l'environnement Minikube (eval \$(minikube docker-env)) pour une disponibilité immédiate.

- **Backend** : Dockerfile Python installant les dépendances ML et exposant le port API.
- **Frontend** : Dockerfile Nginx avec configuration reverse-proxy intégrée.

5.2 Déploiement Kubernetes

Le déploiement est déclaratif via des fichiers YAML versionnés :

- **Namespaces** : Isolation logique (nba, airflow, monitoring).
- **Deployments** : Gestion des pods et redémarrage automatique.
- **Services** : ClusterIP pour la com. interne (Backend, DB), NodePort pour l'accès externe (Frontend).

5.3 Installation Helm (Airflow & Monitoring)

- **Airflow** : Chart officiel configuré avec LocalExecutor. Utilisation d'une base PostgreSQL externe (postgres:16) pour éviter les instabilités des subcharts par défaut.
- **Monitoring** : Stack kube-prometheus-stack. Ajout d'un ServiceMonitor spécifique pour cibler le backend NBA (label release: kube-prom).

5.4 Accès Local

La sécurité et l'accessibilité sont gérées via kubectl port-forward, permettant d'accéder aux interfaces (Airflow UI, Grafana, App NBA) sans exposition publique risquée.

6. Difficultés rencontrées et solutions apportées

L'industrialisation sur un environnement distribué local a présenté plusieurs défis techniques représentatifs.

- **Images Docker introuvables (ImagePullBackOff) :**
 - *Cause* : Docker Desktop et Minikube utilisent des démons différents.
 - *Solution* : Bascule contextuelle via `eval $(minikube docker-env)` et reconstruction des images dans Minikube.
- **Problèmes CORS (Frontend/Backend) :**
 - *Cause* : Appels directs vers `localhost:8080` depuis le navigateur.
 - *Solution* : Implémentation d'un **Reverse Proxy Nginx**. Le frontend appelle `/api/*`, Nginx redirige en interne. Plus de CORS, meilleure portabilité.
- **Installation instable d'Airflow (Helm) :**
 - *Cause* : Incompatibilités des subcharts PostgreSQL et gestion des PVC.
 - *Solution* : Déploiement manuel d'un pod **PostgreSQL officiel** séparé et configuration d'Airflow pour s'y connecter (JDBC).
- **Métriques invisibles (Prometheus) :**
 - *Cause* : Le ServiceMonitor n'était pas détecté par le sélecteur de Prometheus.
 - *Solution* : Ajout du label `release: kube-prom` pour correspondre à la configuration du chart Helm.

7. Modifications apportées au code source initial

Le code initial a été refactorisé pour s'adapter au cloud-native :

- **Backend (FastAPI) :**
 - Ajout de l'endpoint `/metrics` (format Prometheus).
 - Instrumentation du code : compteurs de requêtes et histogrammes de latence ajoutés aux fonctions de prédiction.
- **Frontend** : Suppression des URLs en dur (localhost). Passage à des chemins relatifs pour utilisation derrière le reverse proxy.
- **Infrastructure as Code** : Création intégrale des Dockerfiles, manifestes K8s (Deployments, Services, PVC) et du DAG Airflow `nba_orchestration` pour le pilotage API.

8. Validation et démonstration du bon fonctionnement du pipeline

Cette section valide le fonctionnement de bout en bout via des preuves visuelles.

8.1 Flux Applicatif

L'interface est accessible, la prédiction fonctionne et le résultat s'affiche via le proxy.



N B ACLASSIFICATION

Veuillez entrer le nom du joueur

Découvrir

Ici, nous vous adressons le résultat.

Décision :

RECRUTER OU PAS

Charger un dataset (.csv)

Veuillez entrer des valeurs

turnovers	games played
minutes played	points per game
Field Goals Made	field goal attempts
field goal percent	3 point made
3 point attempts	3 point attempts percent
free throw made	free throw attempts
free throw percent	offensive rebounds
deffensive rebounds	rebounds
assists	steals
Blocks	

Prédire

Réalisé par Ketsia MULAPI – Juin@2021

Figure 3 – Interface utilisateur accessible

Veuillez entrer le nom du joueur

Malik Sealy

Découvrir

Ici, nous vous adressons le résultat.

Décision :

VOUS POUVEZ RECRUTER!

Charger un dataset (.csv)

Figure 4 – Résultat de prédiction affiché

Ce test valide :

- la communication frontend/backend,
- le bon fonctionnement de l'API de prédiction,
- l'intégration du modèle de machine learning dans l'architecture.

8.2 Flux d'Orchestration (Airflow)

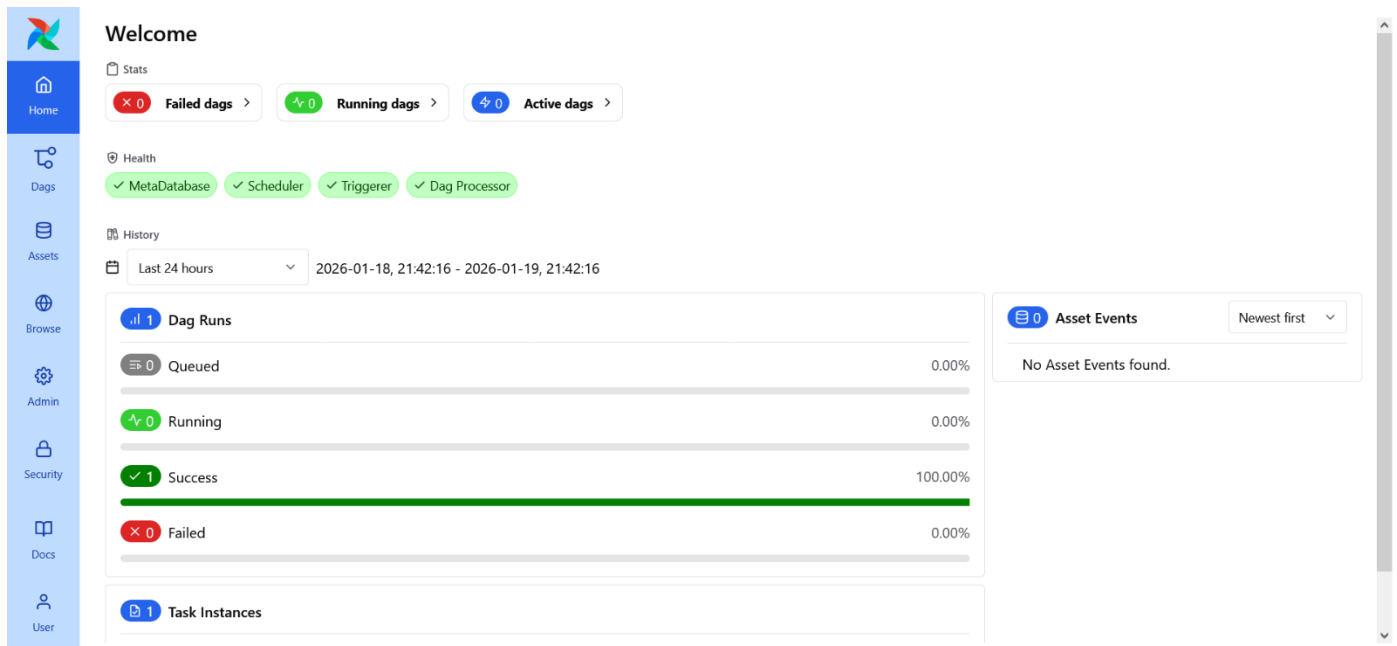
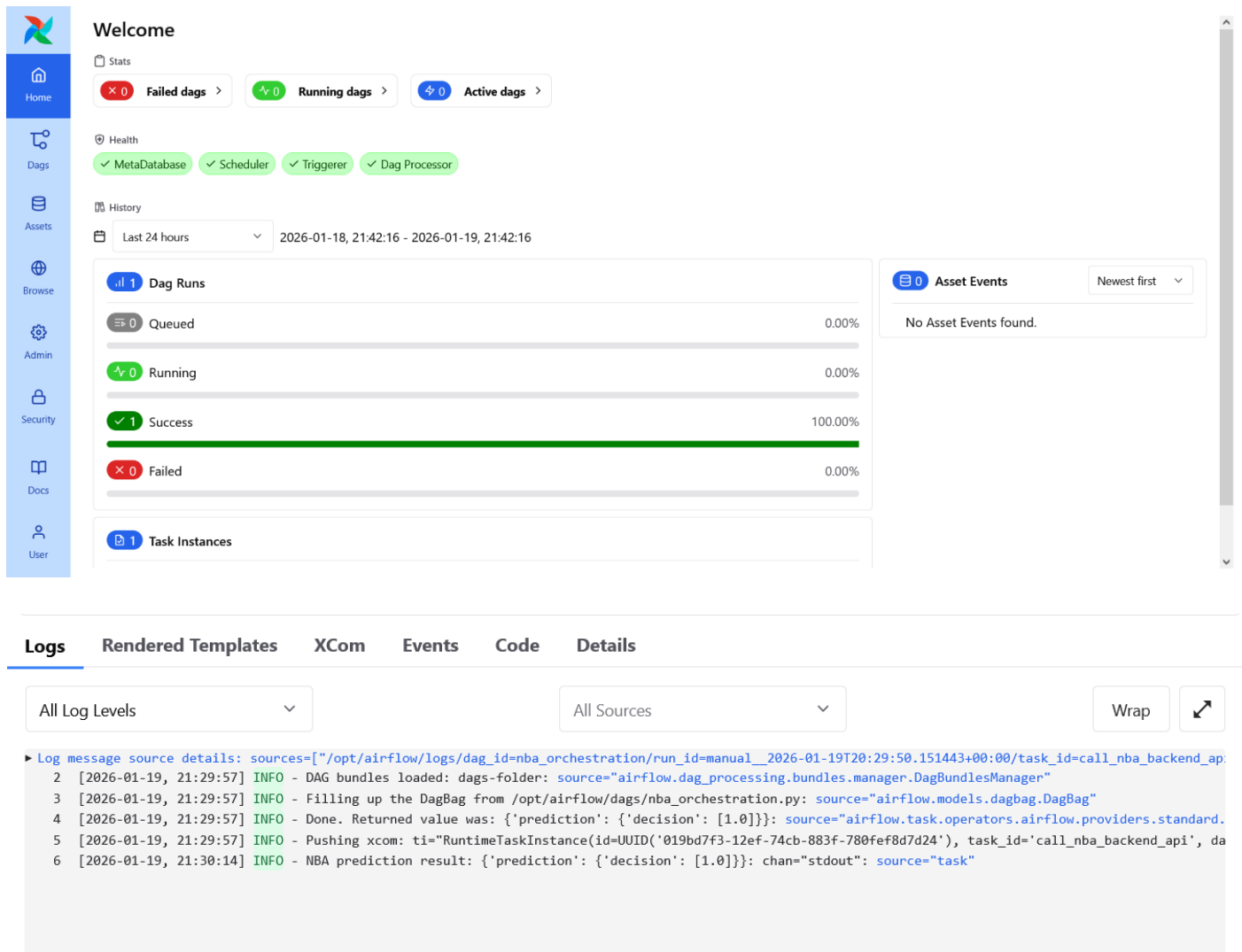


Figure 5 – Interface Airflow



Figures 6.1 et 6.2 – Logs confirmant l'appel API réussi

Ce test valide :

- l'orchestration des traitements via Airflow,
- la communication entre Airflow et l'API backend,
- la capacité à automatiser des appels métier.

8.3 Flux d'Observabilité

Les métriques remontent dans Prometheus et sont historisées dans Grafana.

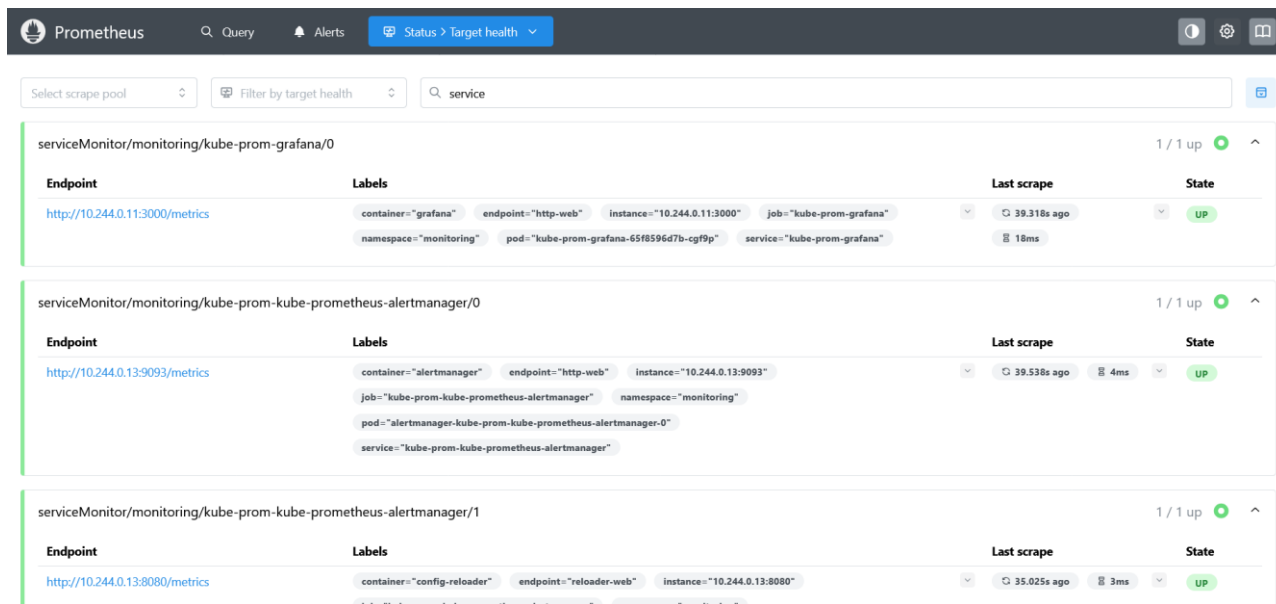
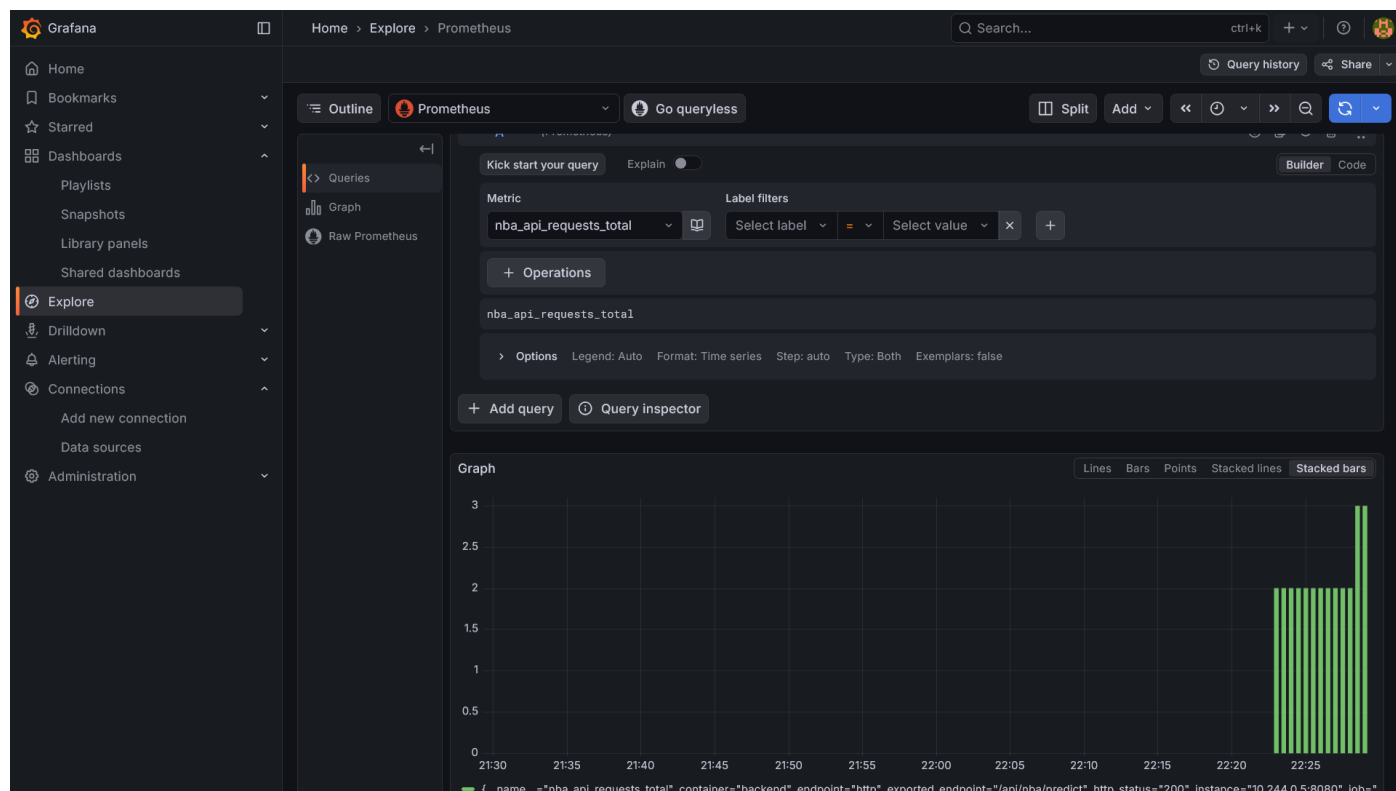
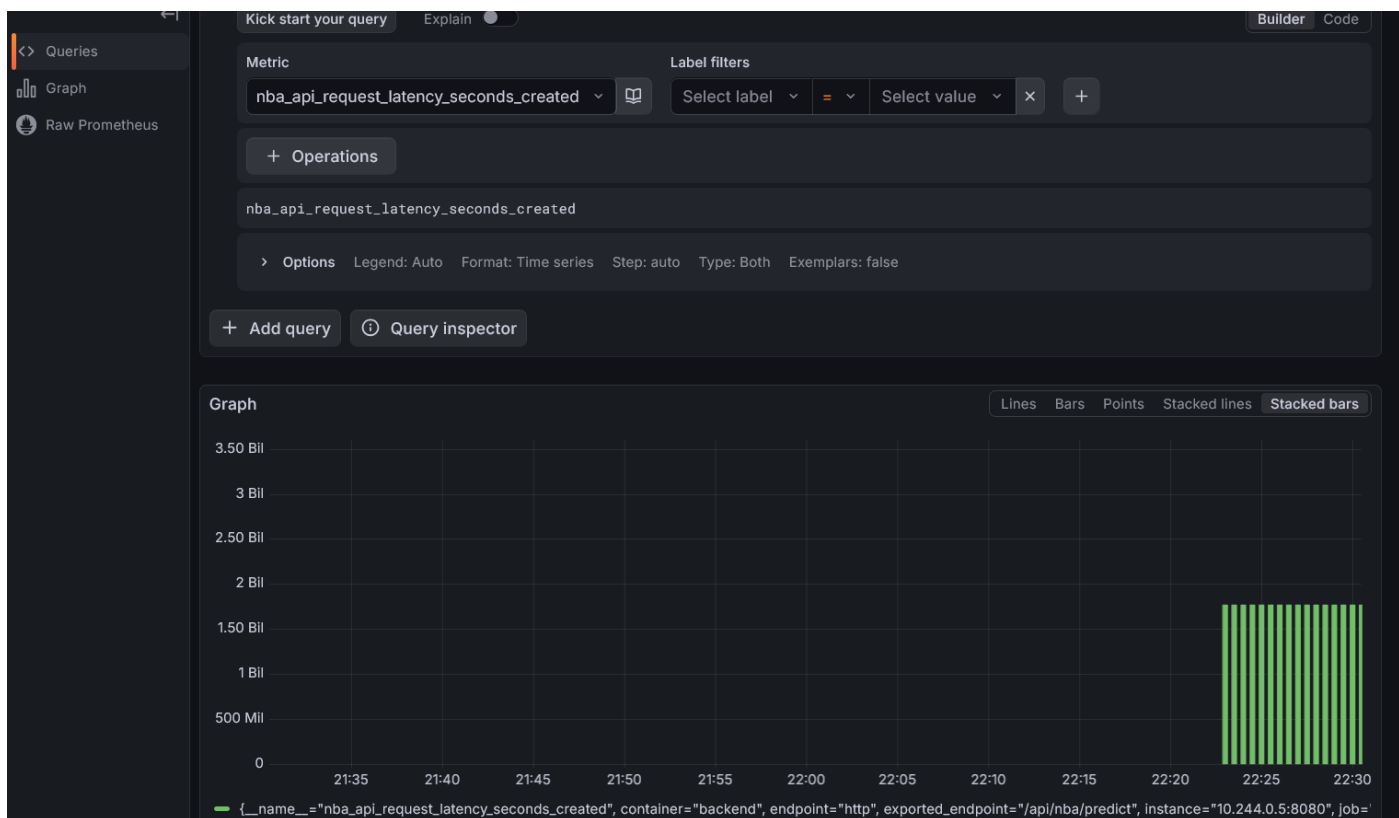


Figure 7 – Target Backend détectée par Prometheus





Figures 8.1 et 8.2 – Dashboard Grafana (Requêtes & Latence)

Ce test valide :

- l'intégration correcte du backend avec Prometheus,
- la visibilité des métriques applicatives,
- l'efficacité du dispositif d'observabilité.

9. Conclusion et perspectives

Ce projet valide la transition d'un modèle ML local vers une **architecture industrielle distribuée**. En intégrant Docker, Kubernetes, Airflow et la stack Prometheus/Grafana, nous avons démontré la capacité à orchestrer et superviser un pipeline de données complexe.

Les difficultés techniques (réseau, persistance, compatibilité Helm) ont renforcé la maîtrise des concepts DevOps. L'architecture est désormais prête pour des évolutions futures :

- **CI/CD** : Automatisation des déploiements (GitLab CI / GitHub Actions).
- **Scalabilité** : Passage à CeleryExecutor pour Airflow et scaling horizontal des pods backend.
- **Cloud** : Migration facilitée vers un cluster managé (GKE/EKS) grâce à l'approche standardisée.

Perspectives d'évolution

Plusieurs pistes d'amélioration et d'extension peuvent être envisagées :

- **Automatisation CI/CD** : mise en place de pipelines de build et de déploiement automatiques (GitHub Actions, GitLab CI).
- **Scalabilité** : passage à un executor Airflow distribué (CeleryExecutor) et montée en charge du backend.
- **Sécurité** : gestion des secrets via Kubernetes Secrets, authentification des API, contrôle des accès aux interfaces.
- **Déploiement cloud** : migration vers un cluster Kubernetes managé (GKE, EKS, AKS).
- **Monitoring avancé** : ajout d'alertes Prometheus et de dashboards métier plus détaillés.

Ces perspectives montrent que l'architecture actuelle constitue une base solide et extensible, adaptée à une montée en complexité progressive.

Pour finir

Ce projet démontre une compréhension globale des enjeux de l'industrialisation des applications de machine learning. Il illustre la capacité à concevoir, déployer et superviser un pipeline complet, en s'appuyant sur des outils modernes et des pratiques alignées avec les standards du secteur.

L'architecture réalisée, bien que déployée en environnement local, est représentative de solutions utilisées en production et constitue une base crédible pour des projets data de plus grande envergure.